Restaurant Order Management Simulator Project Report

Project Team

Student 1 24K-0579

Student 2 24K-0855

Student 3 24K-0721

Session 2024-2028

Course Instuctor

Ms. Rabia Ahmed Ansari

Department of Computer Science

National University of Computer and Emerging Sciences

Karachi, Pakistan

April, 2026

Contents

1. Introduction

This project models a multi-threaded restaurant kitchen system where waiter threads submit customer orders to a shared queue, and chef threads prepare orders concurrently. Mutexes prevent race conditions on shared order queues, while semaphores limit the number of active chefs in the kitchen. This demonstrates producer-consumer synchronization, real-time scheduling, and resource allocation under load. It simulates real-life order management and kitchen optimization in the restaurant industry.

The project utilizes POSIX threads, mutexes, condition variables and semaphores to manage synchronization between threads.

2. Literature Review

The producer consumer problem is a classic problem of synchronization discussed in our course where a producer creates data and multiple consumers perform operations on the data, since the data is shared we must manage access to it safely.

Our project tries to solve the problem implementing priority based processing instead of FIFO, dynamic system behavior and limiting resource usage of threads by using semaphores.

Tools used for POSIX Threads:

pthread_mutex_t used for mutual exclusion

pthread_cond_t used for thread blocking/wakeup

sem_t to limit resources

For priority scheduling simply 1 for VIP and 0 for Normal orders was used.

Instead of using rand() which uses a shared global state which can cause each thread to generate similar random numbers, rand_r() was used which uses local seeds to generate random numbers for each thread.

3. System Design / Architecture

3.1 Overall design

- 3 Waiter Threads which produce orders (Producers).
- 5 Chef Threads which consume orders i.e process them (Consumers).
- Shared Queue for orders.
- Semaphore limits cooking capacity for chefs.

3.2 Data Structures Used

Order struct with id, prep_time, priority, and enqueue_time.

OrderQueue struct containing a vector based queue to manage orders, a mutex lock for synchronization, cond variable to avoid busy wait.

3.3 Shared Variables

- running to control shutdown
- next_id contains the randomly generated unique IDs
- total_completed to store the final statistics
- kitchen_sem used limits chefs
- print_mutex for synchronized output

Locking Discipline makes sure only one lock is held at a time to prevent deadlocks.

4. Implementation

Compilation done through g++ -pthread project.cpp -o project

Priority Queue implemented through a vector list where we iterate through the list to find best order, order is ‘cooked’ by chef if priority is VIP and if all of them have same priority they are ‘cooked’ in order of prep_time.

Waiter Threads:

- Generate Orders
- Assign random priority
- Sleep between orders

Chef Threads:

- Dequeue order
- Wait on semaphore
- Cook (sleep)
- Update stats
- Release semaphores

Safe dequeue done through pthread_cond_timedwait() to avoid deadlocks in shutdown.

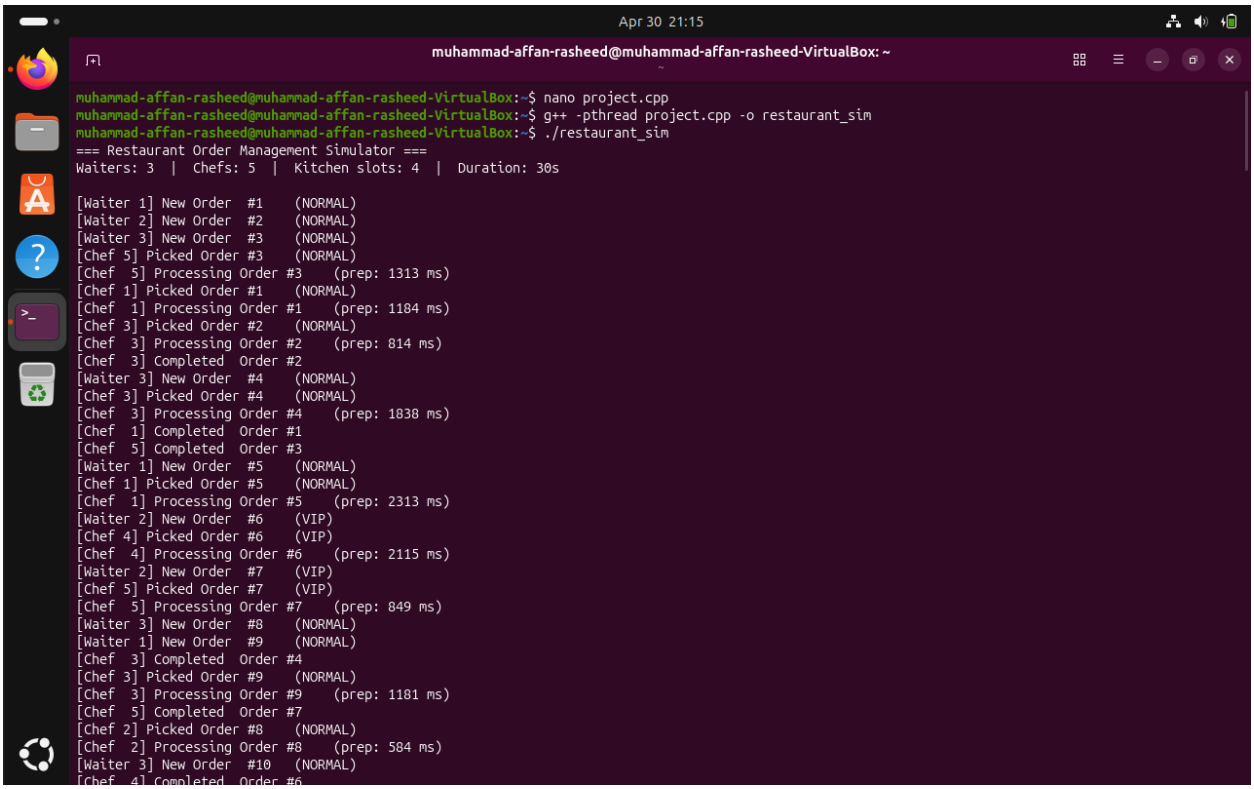
Log_msg() special function was created which uses mutex lock to print log messages to the terminal.

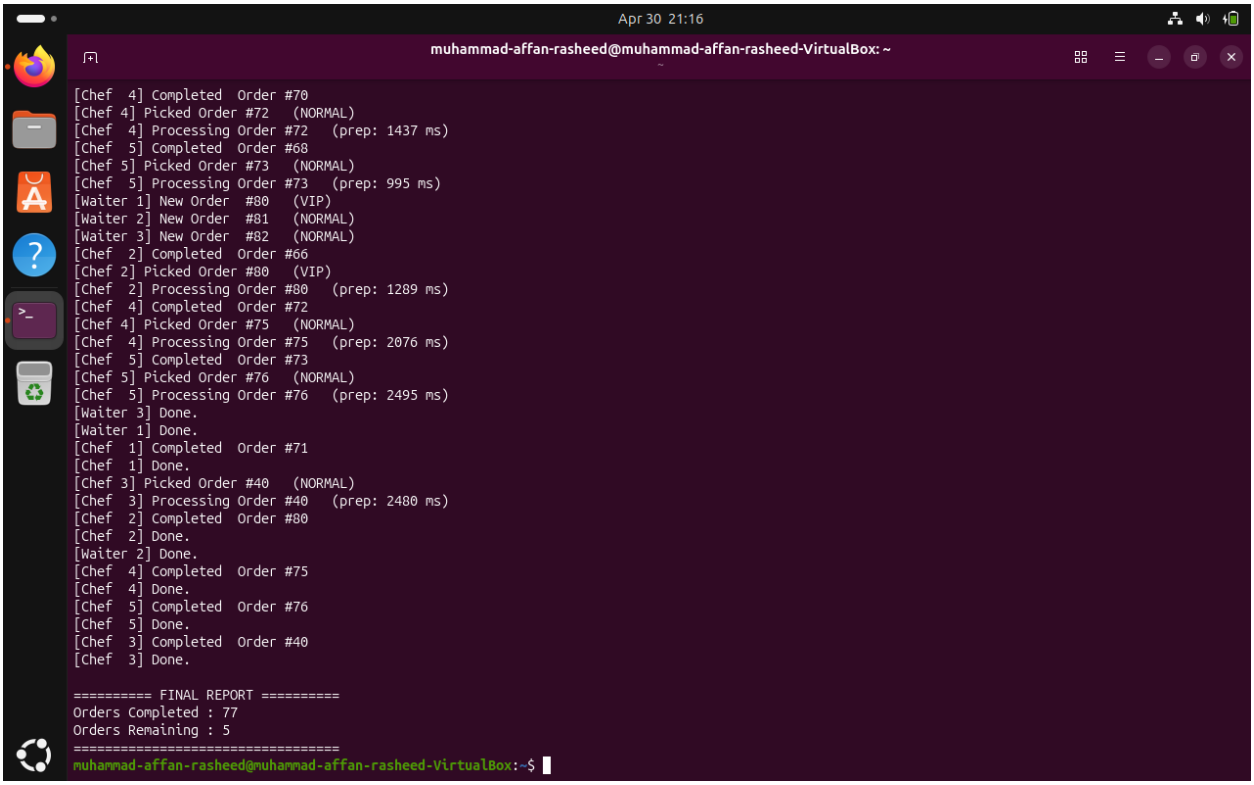
For Shutdown we set running = 0 then broadcast condition variable to wake up any running threads that may be asleep and finally join all threads.

5. Testing and Results

- No major error in compilation and data races were detected.
- VIP orders were processed first as shown in screenshot as order # indicate VIP orders were processed as soon as they arrived.
- Max 4 chefs cooked simultaneously meaning semaphores were implemented correctly
- All threads exited cleanly as show in screenshot meaning program did not exited without problems.
- Rand_R with thread specific seeds made sure no duplicate order ids or similar prep_times.

Screenshots of test run:





6. Discussion

Single file based program and effective synchronization through mutexes and atomic variables worked really well.

Challenges faced were in implementing correct shutdown logic as well as choosing between atomic variables and mutex locks.

Improvements may include using a simple GUI based interface as well as including a Full Stack, live, interactable system with a real time dashboard for an actual restaurant to be used by chefs, waiters and restaurant staff instead of a simple cli based program using simulation. Logs can be exported to a CSV file to allow future inspections and detect problems.

7. Conclusion

The project successfully demonstrates multithreading concepts, synchronization techniques, and safe concurrent programming.

Implementation is efficient, race free and aligns with operating system principles as taught in the course.

8. References

- Learn.microsoft.com
- Course Book
- https://stackoverflow.com/questions/3973665/how-do-i-use-rand-r-and-how-do-i-use-it-in-a-thread-safe-way (rand_r() usage with seeds)
- https://cs.brown.edu/courses/csci0300/2022/notes/l21.html (Synchronization: Atomics, Mutexes, and Condition Variables)
- https://bumbershootsoft.wordpress.com/2022/07/24/custom-printf-wrapping-variadic-functions-in-c/ (helped in creating custom printing log_msg() function with multiple variable arguments)
- https://cppreference-45864d.gitlab-pages.liu.se/en/c/chrono/timespec.html (for timespec usage)